**CGT 390 Lab 9 API and Data Design Report**

Mahina Yasutake | Gatherly | October 22, 2026

**Live prototype:** https://cgt-390-project-seven.vercel.app/

**Repository:** github.com/MahinaYasutake/CGT390_Project

**Release status:** Nine data checks and eleven mobile and analytics checks passed locally. Merged pull request 1 updates the default branch. Vercel deployment and three live task flows passed. Peer testing is pending.

## 1 Interaction and contract

The task is to open Clay Club, save it to a shortlist, and later remove it. This changes stored data in the browser. The implementation uses static HTML and JavaScript; there is no save API endpoint, Server Function, or database.

**Entry point** A document click listener handles buttons with data-save in event-discovery-app/app.js, lines 115-130. event.html renders details; saved.html displays the shortlist.

**Annotated flow** Save button data-save="clay-club" → click listener → toggleSaved(id) → setSaved(ids) → localStorage key gatherly-saved-events. After a successful write, count and page views refresh. On reload, getSaved() reads the same key. These are ordinary browser functions, not separate HTTP endpoints.

**Accepted input** toggleSaved accepts one string matching an ID in the six-event catalog. setSaved accepts an array of known ID strings and removes duplicates. Empty strings, unknown IDs, objects, and non-string IDs are rejected.

**Successful result** toggleSaved returns {ok:true, ids:[...], id:"clay-club", saved:true} when adding and saved:false when removing. The serialized ID list persists in localStorage. The interface reports the new state after the write succeeds.

**Rejected result** An unavailable ID returns {ok:false, error:"This event is not available to save."}. An invalid list returns an error result. A failed write returns a storage-error message and does not show success. There is no save HTTP status to report.

**Code references:** storage and validation lines 10-44; detail lookup and click handler lines 104-130.

## 2 Data and access decisions

**Entities and relationship** Event has a required unique string id, title, date, location, category, organizer, price, and descriptive fields. The current catalog contains six fictional records. A saved list is an array of Event IDs for one browser profile and origin. One list can reference many catalog events; the prototype has no User entity or user_id.

**Important rule** Every saved ID must refer to a catalog event and appear at most once. Save is a toggle: a second activation removes the event. This behavior is intentional and is not an idempotent reservation or repeated save request.

**Source of truth** The events array in app.js defines the public catalog. localStorage is the current browser shortlist store; it is not an authoritative account database. Reads filter unknown and non-string IDs and deduplicate valid IDs. Malformed JSON or a non-array value renders as an empty shortlist without automatically overwriting the damaged value. A later valid write replaces it with a normalized list.

**Access policy** Anonymous visitors may browse and save fictional events. The browser separates localStorage by origin; T3 checks the same application on two different ports. This fits a browser-local shortlist, but it is not user authorization. People sharing the same browser profile and origin share the same shortlist. Client-side code can be bypassed or storage edited directly.

**Public and private fields** All catalog fields are public to anyone who downloads app.js. The saved key stores only event IDs, with no email, token, account ID, or payment information. Any JavaScript executing on that origin can access the list. The save analytics call includes event ID, title, and category; third-party requests were blocked in the tests, so external analytics delivery was not evaluated.

**Continuity from earlier labs** The initial GitHub baseline 4e5a94a lacked the mobile and deferred-analytics changes documented in Lab 7. Existing local accessibility edits were preserved as 51d8641. This release restores second-row mobile navigation, 260px mobile detail artwork, sticky save controls, and interaction-triggered analytics on all four pages. Responsive flows passed at 390, 768, and 1100px; these are new checks, not a rerun of earlier Lighthouse measurements. Lab 8 authentication, database uniqueness, and transactional availability remain proposed architecture.

**Server enforcement** None exists for saving. These repairs improve prototype robustness. Account-backed saves would require server validation, ownership checks, and a unique user-event constraint before the planned Lab 8 architecture could be claimed as implemented.

## 3 Testing evidence

Tests ran on October 22 in headless Google Chrome through Playwright. Each case used a fresh browser context and fictional data. The app was served at http://127.0.0.1:4189; T3 also used port 4191. Predictions were recorded in the runner before execution. The same eight cases ran before and after repair; T9 additionally checked the new function contract.

Identity for every case: anonymous visitor, with no account or role. Results below are browser state and returned values, not save HTTP responses. Full inputs, predictions, actual observations, and pass flags are in the before and after JSON test logs.

### T1 Successful save and remove

**Input and prediction** Save clay-club, reload, open Saved Events, remove it, and reload again. Predict one saved event after saving and zero after removing.

**Observed** Before and after: stored ["clay-club"], count 1, and one saved card. After removal and reload: stored [], count 0. Both runs passed.

**Evidence and interpretation** T1 in both logs. Reload and retrieval verify persistence, rather than relying only on the button label.

### T2 Invalid event identifiers

**Input and prediction** Change the button data-save to not-real and then to an empty string. Predict rejection with unchanged raw storage and a visible error.

**Observed** Before: stored ["not-real",""] and count 2. After: raw storage remained null, count 0, and the page reported "This event is not available to save." Retest passed.

**Evidence and interpretation** T2 in both logs. Validation at toggleSaved, lines 38-43, rejects IDs even when the button attribute is altered.

### T3 Storage access boundary

**Input and prediction** Save at port 4189, open Saved Events at port 4191, then return to 4189. Predict an empty list at the second origin and the original save on return.

**Observed** Both runs: second origin had null storage, count 0, and zero cards; the original retained ["clay-club"]. Passed.

**Evidence and interpretation** T3 in both logs. This demonstrates origin separation in the tested browser, not separation between signed-in users.

### T4 Malformed stored JSON

**Input and prediction** Set the key to {broken and reload. Predict no uncaught error, a rendered detail page, count 0, and unchanged raw storage.

**Observed** Before: JSON parsing threw and detail rendering stopped. After: count 0, Save event rendered, no page errors, and {broken remained stored. Retest passed.

**Evidence and interpretation** T4 in both logs. The guarded parser in getSaved, lines 14-22, prevents a damaged value from crashing the page.

## 3 Testing evidence continued

### T5 Wrong JSON shapes

**Input and prediction** Store {"id":"clay-club"}, reload, then store null and reload. Predict both non-array values render safely as an empty shortlist.

**Observed and interpretation** Before: errors included "getSaved(...).includes is not a function" and a null length error. After: both showed count 0 and Save event with no page errors. Retest passed. T5 in both logs. Array.isArray checks the data shape after JSON parsing; syntactically valid JSON is not necessarily a valid saved list.

### T6 Duplicates and unknown stored IDs

**Input and prediction** Seed ["clay-club","clay-club","not-real",7], reload, then save poster-lab. Predict one valid displayed save and a normalized two-ID write.

**Observed and interpretation** Before: count 4 became 5, retaining duplicates and invalid entries. After: count 1 became 2 and storage was ["clay-club","poster-lab"]. Retest passed. T6 in both logs. getSaved filters valid IDs; Set removes duplicates. The next successful write persists the normalized list.

### T7 Storage write failure

**Input and prediction** Make Storage.prototype.setItem throw QuotaExceededError, then click Save. Predict unchanged storage, no success state, a visible error, and no uncaught page error.

**Observed and interpretation** Before: the exception escaped with no explanatory feedback. After: storage stayed null, count 0, Save event remained, and no page errors occurred. Retest passed. T7 in both logs. setSaved catches write failures before the click handler updates the interface. The visible response was:

> "Could not update saved events. Browser storage may be full or unavailable. Try again."

### T8 Nonexistent event details

**Input and prediction** Open event.html?id=not-real. Predict Event not found, zero save buttons, and unchanged storage.

**Observed and interpretation** Before: Night Market appeared with a save button. After: Event not found, no save button, storage null. Retest passed. The static HTML request returned HTTP 200 in both cases. T8 in both logs. The application-level missing-record state does not change the static host HTTP response.

### T9 Direct function validation

**Input and prediction** Call setSaved with an object, a mixed-type array, and duplicate valid IDs; call toggleSaved with an object. Predict invalid calls fail without mutation and valid duplicates persist once.

**Observed and interpretation** Invalid calls returned ok:false; storage was null after rejected list writes. The valid write returned {ok:true, ids:["clay-club"]} and stored that one ID. Passed. T9 in the after log. Direct browser-function calls check the contract independently of normal button input; there is no server function to invoke.

## 4 Findings and response

The baseline passed 2 of 8 checks. All 8 repeated checks and the additional direct-function check passed after repair. Six failing cases exposed data-handling weaknesses; normal persistence and origin separation already worked.

**Implemented repairs** The code now checks event IDs and list types, safely reads malformed storage, removes duplicate and invalid stored entries from the displayed list, catches storage write errors, and reports failures to the user. Invalid detail IDs show Event not found instead of an unrelated event. Successful UI updates and save analytics occur after a successful storage write.

**Changed code:** current app.js contains the data repairs. Parser and validation: lines 10-44; feedback: lines 46-57; missing event handling: lines 104-110; action handling: lines 115-130. Merged pull request 1 also preserves prior local accessibility edits. Before and after logs provide the retest evidence. The changes are merged into master and deployed through Vercel.

**Release verification** The restored mobile and analytics behavior passed 11 local checks. Three deployed assets matched local source; live task flows passed at 390, 768, and 1100px. Explore now preserves search and filters after saving. Evidence and reproducible test scripts are linked with the results.

**Remaining limitations** This test set does not establish account isolation or server enforcement. It does not cover concurrent-tab writes, storage eviction, all browser engines, or external analytics delivery. Two simultaneous tabs can still overwrite each other through read-modify-write. Malformed storage is preserved until a valid write; it is not repaired by a read alone. Saving still does not reserve a place, guarantee availability, or synchronize to another browser.

## 5 Peer review and AI use

**Peer review status** Pending. No classmate has proposed or run an additional test yet. The blocker is the missing classmate contribution; the nine automated checks are not represented as peer review.

**Classmate handoff** Share the interaction contract and current live prototype. Ask a classmate to propose one additional case beyond T1-T9. A possible discussion starter is two tabs changing the shortlist. Record the peer name or initials, their chosen input, prediction, browser/origin, observed result, and any fix or clarification. Preserve the original result and retest after any repair.

**AI use and verification** ChatGPT/Codex extracted the workshop and previous reports, compared GitHub and local code, wrote the tests, implemented repairs, and drafted this report. Its explanations were checked against source lines, recorded browser errors, returned values, and raw storage before and after reload. Student review and the classmate test remain outstanding; no independent student verification is claimed here.

**Sharing outline** Gatherly stores a browser-local event shortlist. Each saved ID must name one catalog event and appear once. A revealing test stored an unknown ID through an altered button; the repair rejected the action without changing storage. The current boundary is browser origin, while account authorization remains future work.